

A Small Exercise

- Suppose you need to implement a movie rental system
- Your movie rental system must deal with
 - Three kind of movie rental
 - New release (\$3)
 - Regular (\$2)
 - Children (\$2)
 - A movie, of course, can have several copies
 - A customer can ask for a statement
- Please write down
 - the draft classes, some primary methods in each class
 - A main() to show how your classes are used.

1

an example

- The movie rental system
- Please take time to read movie-p05.java
- write down your thoughts
 - Why is it bad from your experience?
 - How you plan to change it? (Compare it with your version, which is better?)



圖 1.1 本例一開始的各個 classes。此圖只顯示最重要的特性。圖中所用符號是 UML (Unified Modeling Language, 統一建模語言, [Fowler, UML])。

2

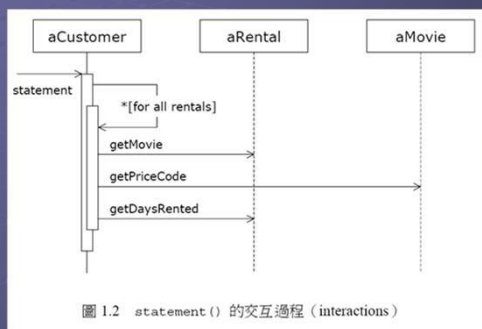


圖 1.2 statement() 的交互過程 (interactions)

3

movie-p05 的問題

- not well designed, poorly written
- hard to change to deal with obvious future evolution.
- **A design is made to deal with focused future change**
- 還記得我曾經告訴你們，沒有任何一個 design 是正確答案。
- 當你要判定一個 design 是否設計良善，你得先看看他所要對付的未來維護需求是什麼

4

未來的維護需求

- 幾乎一定會發生
- 可能會發生
- 會發生但是可能性微乎其微

5

所以?

- 這個例子的幾個主要未來維護需求是什麼？
 - 片子的種類會新增與刪減 (一定會發生)
 - 片子的常客基點與常客的促銷邏輯會更改 (應該會發生)
 - 片子的 print_statement 印出的格式會不斷地修改
 - 片子的價格會改變
 - 一個片子會從新片變成舊片

6

- Image a future change – customer wants a new function called *htmlStatement()*
 1. You will find you cannot reuse any code in p05 formally.
 2. Eventually, you copy-paste **statement()** and modify it into a new **htmlStatement()**
 3. Some time in the future, you need to change the rule of computing frequent renter points
- You need to change two places and maintain the consistency at two places.

7

As time goes by

- As time goes by, programmers come and go and movie-p05 becomes chaotic
- Every fix can be done at once. There are too many places to fix anytime a change request is made by customer because more and more places should remain consistent
- When $\text{cost}(\text{fixing} + \text{debug} + \text{testing}) > \text{cost}(\text{rework})$ it is time to rebuild
- It is called software entropy(熵)
- changes to a bad code is hard

8

Let movie – p05 can deal with Change (The first step)

- Change movie-p05 113-130 into a method of customer
 - Each -> aRental
- Logic : centralize the part of code that is subject to change in the future
- Result is in movie-p11.java

9

Review movie-p11 again

- Do you have any unpleasant feeling about the *amountFor()* in Customer?
- *amountFor* uses two many information from other classes.
- *In most cases, a method should be on the object whose data it uses.*
- We move *amountFor()* to Rental class to become a method *getCharge()*
- remove temp variable *thisAmount*
- see movie-p21.java

10

p19



圖 1.3 搬移「金額計算」函式後，所有 classes 的狀態 (state)

11

p24



圖 1.4 「常客積分計算」函式被提煉及搬移之前的 class diagrams

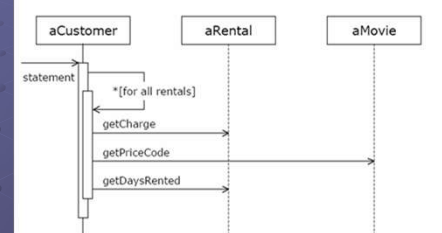


圖 1.5 「常客積分計算」函式被提煉及搬移之前的 sequence diagrams

12

Movie-p21.java

- Now in movie-p21.java, **statement()** becomes much better
- Besides, 101-106 include another part of code that is subject to change
- We change it into another method of Rental class **getFrequentRenterPoints()**
- see movie-p25.java

13

movie-P25.java

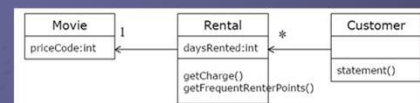


圖 1.6 「常客積分計算」函式被提煉及搬移之後的 class diagram

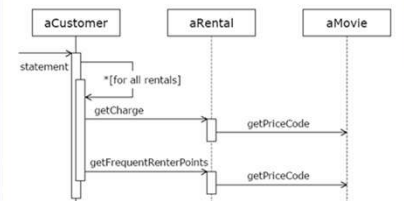


圖 1.7 「常客積分計算」函式被提煉及搬移之後的 sequence diagram

14

p30

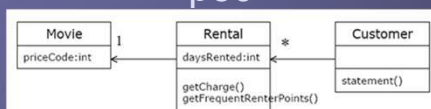


圖 1.8 「總量計算」函式被提煉前的 class diagram

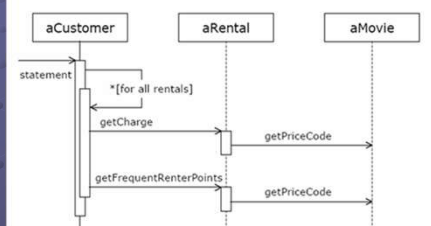


圖 1.9 「總量計算」函式被提煉前的 sequence diagram

15

Remove temp variables

- temp variables can be a problem, which encourage long routines
- remove temps totalAmounts and frequentRenterPoints
- see movie-p33.java
- In movie-p33.java a htmlStatement() method is added. -> now if we want to change rule, only one place should be changed.

16

movie-p33

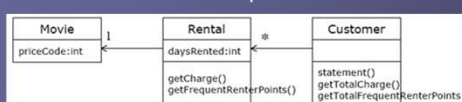


圖 1.10 「總量計算」函式被提煉後的 class diagram

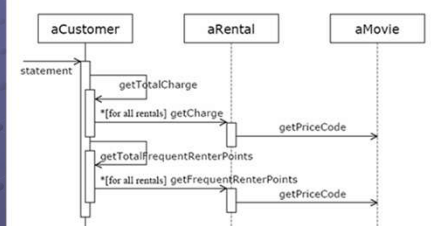


圖 1.11 「總量計算」函式被提煉後的 sequence diagram

17

Review movie-p33.java

- Is this code perfect now? not yet
- It is a bad idea to do a switch based on an attribute of another object
- For example, when a new movie class called Adult is added, changes are
 - Change the definition in movie class.
 - Rental::getCharge() should be changed
- Let's move getCharge(), getFrequentRenterPoints() 的 code 搬到 movie class
- see movie-p37.java

18

p36

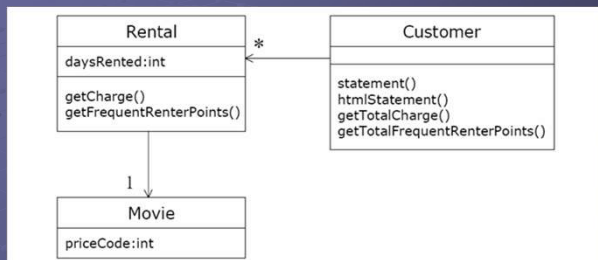


圖 1.12 本節所討論的兩個函式被移到 Movie class 內之前，系統的 class diagram。

19

19

p37

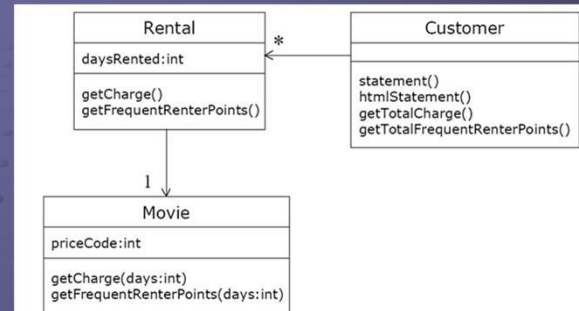


圖 1.13 本節所討論的兩個函式被移到 Movie class 內之後，系統的 class diagram。

20

20

Review of movie-p37

- So, how about it now? perfect? not yet
- switch in getCharge() is a typical bad code which cannot answer changes well.
- In practice, such switch() can occurs in several places (recall that when polymorphism is introduced). Each switch must be kept consistent.
- Often, a switch that will change from time to time satisfies the precondition of subclassing. We can replace it by polymorphism and inheritance

21

21

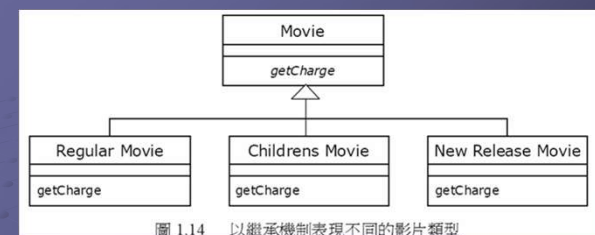


圖 1.14 以繼承機制表現不同的影片類型

- Move each branch of switch to subclasses getCharge 中
- We replace switch statement with polymorphism
- Feeling good? (自我感覺良好嗎?)

22

22

- Unfortunately, such inheritance structure has flaw. However, amateur OO analysts will most reach such kind of results
- In this application, a movie object may change its classification during life time
 - E.g., suppose a new release film Q changes to a regular movie
 - Consequently, what will you do?
 - New a regular movie P
 - Clone the data of Q to P
 - Destroy Q
 - That's it? (No. take a look at the main())
- An object cannot change its class during life time

23

23

state design pattern

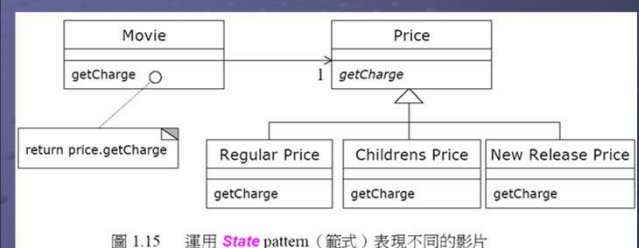


圖 1.15 運用 State pattern (範式) 表現不同的影片

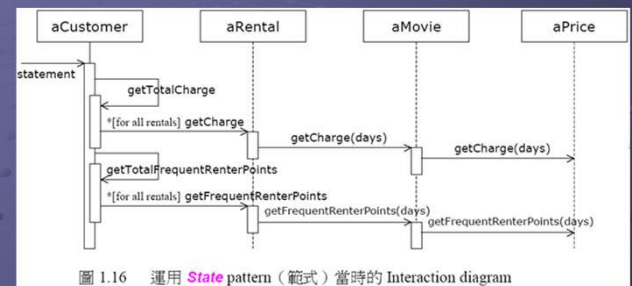
24

24

- The correct answer is change movie-p37 to movie-p52.java using state design pattern
- Please see movie-p52.java

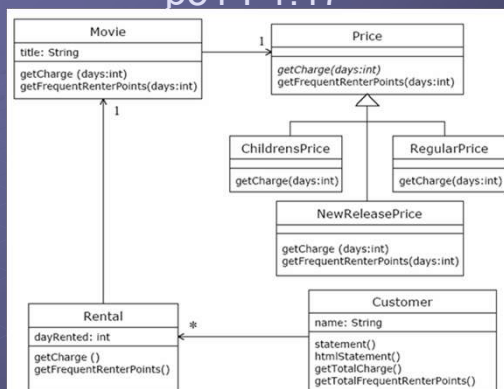
25

p51 Fig1.16

圖 1.16 運用 *State* pattern (範式) 當時的 Interaction diagram

26

p51 F1.17

圖 1.17 加入 *State* pattern (範式) 之後的 class diagram

27

Refactoring (重構)

- The above technique is called refactoring, a mechanism to improve architecture of as-build codes.
- change is based on step by step and each step is validated by all the testing cases.
- It is of course, not cost-free

28

Design Patterns

- 過去，在實做各種各類的軟體過程中，許多人累積了寶貴的物件導向分析經驗。經過蒐集整理，這些寶貴的繼承架構，class diagram，物件互動架構等等，被蒐集成所謂的 design patterns。
- When you encounter some problems in OOAD or OOP. It is rare that your problem is new.
- Some people collect these problems and solution into a some form of reuses.
- design patterns are one kind of reuse, but not code reuse. Instead, it is a kind of pattern reuse.

29

Discussion

- The main purpose of OOAD is to make our code easy to maintain, change, and evolve,
- Good analysis is difficult
- Don't expect you will become a OOAD expert because you take the course.
- Typically, having analysis is better than no analysis. Recall the integrity of architecture and thought.

30

Discussion

- Good architecture != good performance
- In most cases, good architecture may have worse performance.
- In some design concerns, some classes will be merged to increase performance (if they are found to be guilty of the blame)
- Programmers that can write good and clean OO code are hard to find.
- Good analysis, of course, can be used to create good teamwork (WBS work breaking structure)
- Good analyst can smell the change in system design and make the part OO ly
- Don't overdesign. If the change will not occurs within 100 years, make the part Ooly is a waste of time.
- Good OOAD must eventually demonstrate at the level of code. Extensive programming experience is required.

31

Discussion

- In a software development project. How much efforts should be put in OOAD?
 - What type of software you build (what kind of market you are in)?
 - e.g. there is no need for analysis for most research
 - Is evolveability very important in your area?
 - is technology changing very fast in your area?
 - How long is your design/code typically out of date and thrown away?
 - what is the scale of your software?
 - what is the total cost of your software?
 - Are documentation/process important in your company?
 - Have your programmers high transition rate?
 - How much quality you care?
 - 6 month design/planning, 3 month coding, 3 month testing

32

Thanks

- Q & A

33